

Multimodal RAG Information Retrieval System

Apple Inc. 2022 Q3 10-Q — Technical Design & Methodology Report

Candidate: Harsh Raj

Target role: Software Engineer I, Machine Learning

1. Executive Summary

This submission implements a production-oriented Retrieval-Augmented Generation (RAG) system designed specifically for a financial 10-Q document. The central design decision is to treat narrative text, tables, and figures as different but interoperable retrieval modalities. The pipeline extracts structured evidence, indexes it using both lexical and semantic retrieval, reranks candidates, and generates a grounded answer with page-level citations. The system also supports an abstention path when the retrieved evidence is insufficient.

The implementation is deliberately modular: ingestion, retrieval, generation and API layers are independent. This makes it straightforward to replace local FAISS/BM25 with a managed search platform, add authentication and observability, or scale ingestion through asynchronous workers without redesigning the user-facing contract.

2. Requirements and Alignment

Assessment requirement	Implementation
Text content	PyMuPDF text extraction + semantic/lexical retrieval
Tables	pdfplumber table extraction; rows preserved as structured text
Figures	Page rendering + optional vision-model descriptions
RAG	Hybrid retrieval → reranking → grounded LLM generation
Maintainability	Typed models, configuration, modular services, tests, Docker
Evidence	Page/modality/source metadata returned with every hit

3. End-to-End Architecture

Step 1 — Document ingestion: the source PDF is parsed page-by-page. Text is normalized while preserving page numbers. Tables are extracted separately so their row/column relationships are not destroyed by ordinary text extraction.

Step 2 — Figure processing: pages containing embedded images are rendered to PNG. An optional vision model produces a conservative textual description. The original image path and page number remain in metadata, so a future UI can display the visual evidence itself.

Step 3 — Chunking: narrative text is split into overlapping word windows. Table chunks remain table-oriented. This avoids using a single generic chunking strategy for every modality.

Step 4 — Indexing: chunks are represented in a unified schema containing document ID, page, modality, source and optional image path. Dense embeddings are stored in FAISS and lexical tokens are indexed by BM25.

Step 5 — Retrieval: a query is sent to both dense and lexical retrieval. Their candidate sets are merged, then a cross-encoder reranks the combined evidence. This is important for financial documents where exact strings such as 'net sales', '\$81,797', fiscal dates and accounting terms are often as important as semantic similarity.

Step 6 — Grounded generation: only the top evidence is passed to the LLM. The system prompt requires evidence-only answers, explicit uncertainty when evidence is insufficient, and page-level citations.

4. Why Hybrid Retrieval?

Dense-only retrieval is strong for paraphrases but can underweight exact financial terminology and numeric expressions. BM25 provides an independent lexical signal. Rather than adding the two scores directly—which would be unsafe because their numeric scales differ—the implementation uses both methods to construct a candidate pool and delegates final ordering to a cross-encoder reranker.

This architecture also makes debugging easier: a retrieval hit can be traced to a concrete chunk, page and modality rather than being an opaque vector-store result.

5. Table Retrieval Strategy

Financial tables are treated as first-class chunks. Each extracted row is serialized with column boundaries using a pipe-delimited representation. This keeps values associated with their headers and makes questions such as 'What was total net sales for the nine months?' more reliable than flattening the entire page into one text block.

For production, the next iteration would persist table cells with row/column coordinates and add a table-aware parser for merged cells, footnotes and multi-page tables. Numerical questions could then be routed through a deterministic calculation layer before generation.

6. Figure Retrieval Strategy

Figures are difficult for conventional text RAG because their information may exist primarily in pixels. The pipeline therefore renders the relevant page and can create a vision-derived description. The description becomes searchable text while the source image remains available for inspection.

A production deployment should store a figure identifier, bounding box and image artifact separately rather than relying only on the page image. This would support precise visual citations and lower vision-model cost by captioning only actual figure regions.

7. Grounding, Hallucination Control and Citations

- The generation prompt explicitly restricts answers to retrieved evidence.
- Numerical questions are instructed to prefer table evidence.
- The model is instructed to abstain when the evidence is insufficient.
- Every material claim should include a page/modality citation.
- Retrieval metadata is returned to the caller so answers can be audited.

8. API Contract

POST /query

```
{"question":"What was Apple's net sales for the nine months ended June 25, 2022?"}
```

The response contains answer, latency_ms, and an evidence array containing the source chunk, page, modality, retrieval method and score. GET /health exposes a lightweight readiness contract.

9. Project Structure

app/core/models.py – shared retrieval data contracts
app/core/ingest.py – PDF parsing, chunking, tables, figures
app/core/retrieval.py – BM25 + FAISS + reranking
app/core/generation.py – grounded LLM and vision calls
app/core/pipeline.py – orchestration
app/api/main.py – FastAPI service
scripts/ingest.py – ingestion entry point
tests/ – unit/API contract tests
Dockerfile / docker-compose.yml – container deployment

10. Assumptions

- The supplied assessment PDF is the authoritative source for answering document questions.
- An OpenAI-compatible API key is available for final generation and optional figure captioning.
- The initial exercise uses one PDF, but the schema includes document_id so multiple documents can be added.
- The local vector index is suitable for the take-home dataset; a managed search service is recommended for production scale.

11. Challenges and Mitigations

Challenge	Mitigation
Numbers appear in multiple places	Hybrid retrieval + reranking + page provenance
Tables lose meaning when flattened	Dedicated table extraction and modality metadata
Figures are not ordinary text	Page rendering + optional vision descriptions
LLM hallucination risk	Evidence-only prompt + abstention + citations
Changing model/vendor	Configuration-driven model names and isolated generation module
Deployment reproducibility	requirements.txt + Dockerfile + docker-compose

12. Production Hardening Roadmap

- **Search:** replace local FAISS/BM25 with Azure AI Search, OpenSearch or Elasticsearch and use hybrid/vector + BM25 in one managed index.
- **Scale:** asynchronous ingestion workers, object storage for page/figure artifacts, and document-level versioning.
- **Reliability:** retries with backoff, circuit breakers, request timeouts and graceful model fallbacks.
- **Security:** secret manager, API authentication, authorization, input limits and prompt-injection defenses.
- **Observability:** structured logs, traces, retrieval hit-rate, groundedness evaluation, latency and token-cost dashboards.
- **Evaluation:** create a curated question set covering text, table arithmetic, cross-page reasoning and figure interpretation; measure Recall@K, MRR, answer groundedness and citation accuracy.

13. Evaluation Plan

A serious deployment should not evaluate only whether an answer 'sounds correct'. Retrieval and generation should be evaluated separately. Retrieval metrics include Recall@K and MRR against labeled evidence pages. Generation

metrics include answer correctness, citation correctness, groundedness and abstention quality. A small regression suite should run in CI for every change to chunking, embedding models, prompts or reranking.

14. Role Alignment

The supplied Software Engineer I, Machine Learning description emphasizes building and improving ML systems, LLMs/RAG, Python or another modern programming language, Git/version control, Unix-like systems, basic containerization, process analysis, documentation and disciplined delivery. This project demonstrates those areas through an end-to-end, runnable RAG service rather than only a notebook prototype. ■filecite■turn0file0■L20-L33■

15. Submission Checklist

- GitHub repository containing the complete source code.
- Separate PDF methodology/design report.
- README with setup, ingestion and API execution instructions.
- Environment template without committed secrets.
- Automated tests for core preprocessing and API contract.
- Docker deployment files.
- Clear explanation of text/table/figure retrieval and grounded generation.

Source document: Apple Inc. 2022 Q3 10-Q PDF supplied in the assessment. The repository intentionally does not commit the source PDF or API secrets; the README explains how to place the supplied document under data/ before ingestion.